Assignment 9: Advanced SQL Queries Using Joins and Subqueries

DBMS Assignment Series | Project topic: **Online Food Delivery System (QuickBite)**

Name: **Jagan S** | Roll No.: **2501121240031**

1. Objective

Develop advanced reports for the QuickBite system using joins (INNER, LEFT) across multiple tables, summary statistics, and subqueries. Each report lists the SQL, the real output and a brief explanation.

Note: Order value = sum of (quantity x unit_price) of its lines, plus delivery_fee. Executed on SQLite; in MySQL replace `strftime('%Y-%m', order_date)` with `DATE_FORMAT(order_date, '%Y-%m')`.

Section 2: Reports using multiple joins

Report 1. Orders in progress (4-table join) - Placed orders with customer, restaurant and delivery partner.

```
SELECT o.order_id, c.full_name AS customer, r.restaurant_name AS restaurant,
       p.partner_name AS partner, o.order_date, o.payment_method
FROM orders o
JOIN customers c    ON c.customer_id   = o.customer_id
JOIN restaurants r  ON r.restaurant_id = o.restaurant_id
LEFT JOIN delivery_partners p ON p.partner_id = o.partner_id
WHERE o.status = 'Placed'
ORDER BY o.order_date;
```

Output (3 rows):

order_id	customer	restaurant	partner	order_date	payment_method
24	Mohan Raj	Dosa Corner	Suresh Babu	2026-10-22	UPI
25	Aditya Rao	Spice Garden	Manju Nath	2026-10-24	Card
26	Harish Gowda	Dragon Bowl	Ravi Kumar	2026-10-25	UPI

Explanation: Four tables are linked through foreign keys. LEFT JOIN is used for the partner so an order without a partner would still be listed.

Report 2. Bill of every delivered order - Item total, delivery fee and bill amount (top 10 by value).

```
SELECT o.order_id, c.full_name AS customer, r.restaurant_name AS restaurant,
       SUM(oi.quantity * oi.unit_price) AS items_total,
       o.delivery_fee,
       SUM(oi.quantity * oi.unit_price) + o.delivery_fee AS bill_amount
FROM orders o
JOIN customers c    ON c.customer_id   = o.customer_id
JOIN restaurants r  ON r.restaurant_id = o.restaurant_id
JOIN order_items oi ON oi.order_id     = o.order_id
WHERE o.status = 'Delivered'
GROUP BY o.order_id, c.full_name, r.restaurant_name, o.delivery_fee
ORDER BY bill_amount DESC
LIMIT 10;
```

Output (10 rows):

order_id	customer	restaurant	items_total	delivery_fee	bill_amount
17	Harish Gowda	Spice Garden	860	0	860
5	Pooja Shetty	Spice Garden	800	0	800
15	Nisha Verma	Pizza Palace	727	40	767
4	Aditya Rao	Dragon Bowl	730	30	760
1	Aditya Rao	Spice Garden	720	30	750
8	Harish Gowda	Pizza Palace	698	0	698
3	Rahul Menon	Pizza Palace	626	40	666
14	Fatima Sheikh	Spice Garden	560	30	590
10	Ishita Das	Sweet Tooth	470	30	500

order_id	customer	restaurant	items_total	delivery_fee	bill_amount
12	Mohan Raj	Dragon Bowl	470	30	500

Explanation: Joins the order lines to their order and aggregates per order, producing the bill the customer paid.

Report 3. Detailed bill of one order - Line-by-line items of order 17.

```
SELECT oi.order_id, mi.item_name, mi.category, oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_total
FROM order_items oi
JOIN menu_items mi ON mi.item_id = oi.item_id
WHERE oi.order_id = 17
ORDER BY mi.item_name;
```

Output (3 rows):

order_id	item_name	category	quantity	unit_price	line_total
17	Dal Makhani	Main Course	1	240	240
17	Gulab Jamun	Dessert	2	90	180
17	Paneer Tikka	Starter	2	220	440

Explanation: The junction table order_items is joined to menu_items to replace item ids with readable dish names.

Section 3: Outer joins (related records that may be missing)

Report 4. LEFT JOIN - customer spending - Every customer with delivered orders and food spend, including customers with none.

```
SELECT c.customer_id, c.full_name, c.membership,
       COUNT(DISTINCT o.order_id)                AS delivered_orders,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS food_spend
FROM customers c
LEFT JOIN orders o       ON o.customer_id = c.customer_id AND o.status = 'Delivered'
LEFT JOIN order_items oi ON oi.order_id   = o.order_id
GROUP BY c.customer_id, c.full_name, c.membership
ORDER BY food_spend DESC, c.customer_id;
```

Output (12 rows):

customer_id	full_name	membership	delivered_orders	food_spend
1	Aditya Rao	Regular	4	2069
7	Harish Gowda	Premium	2	1558
4	Pooja Shetty	Premium	3	1378
3	Rahul Menon	Regular	3	1225
2	Nisha Verma	Regular	2	1027
8	Ishita Das	Regular	2	880
6	Tara Nair	Regular	2	630
10	Fatima Sheikh	Regular	1	560
9	Mohan Raj	Regular	1	470
5	Sanjay Kumar	Regular	1	150
11	Kiran Hegde	Regular	0	0
12	Leela Pillai	Regular	0	0

Explanation: LEFT JOIN keeps all customers; COALESCE turns NULL sums into 0. Leela Pillai (never ordered) and Kiran Hegde (only a cancelled order) show 0.

Report 5. LEFT JOIN - dishes never ordered - Menu items that no customer has ordered.

```
SELECT mi.item_id, mi.item_name, r.restaurant_name, mi.price
FROM menu_items mi
JOIN restaurants r ON r.restaurant_id = mi.restaurant_id
LEFT JOIN order_items oi ON oi.item_id = mi.item_id
WHERE oi.order_item_id IS NULL
ORDER BY mi.item_id;
```

Output (2 rows):

item_id	item_name	restaurant_name	price
8	Chicken Chettinad	Dosa Corner	280
24	Mango Smoothie	Sweet Tooth	120

Explanation: Rows with no match on the right side have NULL in oi columns; filtering on IS NULL finds the unsold dishes - useful for menu clean-up.

Section 4: Summary statistics across tables

Report 6. Restaurant-wise sales report - Delivered orders, units sold and revenue per restaurant.

```
SELECT r.restaurant_name, r.cuisine,
       COUNT(DISTINCT o.order_id)        AS delivered_orders,
       SUM(oi.quantity)                  AS units_sold,
       SUM(oi.quantity * oi.unit_price)  AS revenue
FROM restaurants r
JOIN orders o       ON o.restaurant_id = r.restaurant_id
JOIN order_items oi ON oi.order_id     = o.order_id
WHERE o.status = 'Delivered'
GROUP BY r.restaurant_id, r.restaurant_name, r.cuisine
ORDER BY revenue DESC;
```

Output (6 rows):

restaurant_name	cuisine	delivered_orders	units_sold	revenue
Spice Garden	North Indian	5	16	3350
Pizza Palace	Italian	4	10	2450
Dragon Bowl	Chinese	3	9	1610
Dosa Corner	South Indian	5	14	1070
Burger Hub	Fast Food	2	6	737
Sweet Tooth	Desserts	2	5	730

Explanation: Joins three tables, filters on delivered orders and groups by restaurant - a revenue ranking for the business team.

Report 7. Delivery partner performance - Deliveries completed and fees handled by each partner (including idle partners).

```
SELECT p.partner_name, p.vehicle, p.availability,
       COUNT(o.order_id)             AS deliveries,
       COALESCE(SUM(o.delivery_fee), 0) AS delivery_fees
FROM delivery_partners p
LEFT JOIN orders o ON o.partner_id = p.partner_id AND o.status = 'Delivered'
GROUP BY p.partner_id, p.partner_name, p.vehicle, p.availability
ORDER BY deliveries DESC, p.partner_name;
```

Output (5 rows):

partner_name	vehicle	availability	deliveries	delivery_fees
Ravi Kumar	Bike	Busy	7	190
Suresh Babu	Scooter	Busy	6	120
Manju Nath	Bike	Busy	5	100
Deepak Joshi	Cycle	Available	3	90
Anil Reddy	Bike	Offline	0	0

Explanation: LEFT JOIN keeps partners with no deliveries (Anil Reddy, offline) so the report is complete.

Report 8. Top-selling dishes - Five best-selling dishes by units in non-cancelled orders.

```
SELECT mi.item_name, r.restaurant_name, SUM(oi.quantity) AS units_sold
FROM order_items oi
JOIN orders o       ON o.order_id = oi.order_id
JOIN menu_items mi  ON mi.item_id = oi.item_id
JOIN restaurants r  ON r.restaurant_id = mi.restaurant_id
WHERE o.status <> 'Cancelled'
GROUP BY mi.item_id, mi.item_name, r.restaurant_name
ORDER BY units_sold DESC, mi.item_name
LIMIT 5;
```

Output (5 rows):

item_name	restaurant_name	units_sold
Masala Dosa	Dosa Corner	7
Butter Chicken	Spice Garden	5
Dal Makhani	Spice Garden	5
Filter Coffee	Dosa Corner	5
Gulab Jamun	Spice Garden	5

Explanation: Four tables are joined; cancelled orders are excluded; dishes are ranked by total quantity sold.

Report 9. Monthly sales report - Orders and food revenue per month for delivered orders.

```
SELECT strftime('%Y-%m', o.order_date) AS month,
       COUNT(DISTINCT o.order_id)       AS delivered_orders,
       SUM(oi.quantity * oi.unit_price) AS food_revenue,
       ROUND(SUM(oi.quantity * oi.unit_price) * 1.0 / COUNT(DISTINCT o.order_id), 2) AS avg_order_value
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status = 'Delivered'
GROUP BY month
ORDER BY month;
```

Output (3 rows):

month	delivered_orders	food_revenue	avg_order_value
2026-08	5	3176	635.2
2026-09	8	3317	414.63
2026-10	8	3454	431.75

Explanation: Groups orders by year-month and shows revenue and average order value, which helps track growth over time.

Section 5: Subqueries

Report 10. Scalar subquery in WHERE - Dishes priced above the average dish price.

```
SELECT item_name, price
FROM menu_items
WHERE price > (SELECT AVG(price) FROM menu_items)
ORDER BY price DESC;
```

Output (9 rows):

item_name	price
Pepperoni Pizza	399
Butter Chicken	320
Margherita Pizza	299
Chicken Chettinad	280
Chilli Chicken	260
Dal Makhani	240
Paneer Tikka	220
Hakka Noodles	210
Chicken Burger	180

Explanation: The inner query returns one value (the average price) which the outer query uses for comparison.

Report 11. Subquery with IN - Customers who have ordered from 'Dosa Corner'.

```
SELECT full_name, area
FROM customers
WHERE customer_id IN (
    SELECT o.customer_id
    FROM orders o
    JOIN restaurants r ON r.restaurant_id = o.restaurant_id
    WHERE r.restaurant_name = 'Dosa Corner')
ORDER BY full_name;
```

Output (6 rows):

full_name	area
Aditya Rao	Indiranagar
Mohan Raj	Whitefield
Nisha Verma	Koramangala
Rahul Menon	HSR Layout
Sanjay Kumar	Whitefield
Tara Nair	Koramangala

Explanation: The inner query returns the ids of customers who ordered from that restaurant; the outer query returns their details.

Report 12. Subquery with NOT EXISTS - Customers who have never placed an order.

```
SELECT full_name, signup_date
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Output (1 row):

full_name	signup_date
Leela Pillai	2026-10-21

Explanation: A correlated subquery - evaluated per customer, true only when no order row exists for that customer.

Report 13. Subquery in HAVING - Customers who placed more orders than the average per ordering customer.

```
SELECT customer_id, COUNT(*) AS orders_placed
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > (SELECT AVG(cnt) FROM (SELECT COUNT(*) AS cnt FROM orders GROUP BY customer_id))
ORDER BY orders_placed DESC, customer_id;
```

Output (4 rows):

customer_id	orders_placed
1	5
3	3
4	3
7	3

Explanation: The innermost query counts orders per customer, the middle one averages those counts, and HAVING keeps the above-average customers.

Report 14. CTE with subquery - The delivered order with the highest bill amount.

```
WITH order_totals AS (
    SELECT o.order_id, SUM(oi.quantity * oi.unit_price) + o.delivery_fee AS bill_amount
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status = 'Delivered'
    GROUP BY o.order_id, o.delivery_fee)
SELECT order_id, bill_amount
FROM order_totals
WHERE bill_amount = (SELECT MAX(bill_amount) FROM order_totals);
```

Output (1 row):

order_id	bill_amount
17	860

Explanation: A named temporary result (CTE) holds every bill; the subquery finds the maximum, and ties would all be reported.

Report 15. Correlated subquery - Dishes priced above the average of their own restaurant's menu.

```
SELECT r.restaurant_name, mi.item_name, mi.price
FROM menu_items mi
JOIN restaurants r ON r.restaurant_id = mi.restaurant_id
WHERE mi.price > (SELECT AVG(price) FROM menu_items WHERE restaurant_id = mi.restaurant_id)
ORDER BY r.restaurant_name, mi.price DESC;
```

Output (12 rows):

restaurant_name	item_name	price
Burger Hub	Chicken Burger	180
Burger Hub	Chocolate Shake	140
Dosa Corner	Chicken Chettinad	280
Dragon Bowl	Chilli Chicken	260
Dragon Bowl	Hakka Noodles	210
Pizza Palace	Pepperoni Pizza	399
Pizza Palace	Margherita Pizza	299
Spice Garden	Butter Chicken	320
Spice Garden	Dal Makhani	240
Spice Garden	Paneer Tikka	220
Sweet Tooth	Ice Cream Sundae	170
Sweet Tooth	Chocolate Brownie	150

Explanation: The inner query refers to the outer row (mi.restaurant_id), so the average is recalculated for each restaurant.